

Lab Assignment # 10

Course Title : **AI Assistant Coding**
Name of Student : **BG Sreevani**
Enrollment No. : **2303A54066**
Batch No. : **48**

Lab 10 – Code Review and Quality: Using AI to improve code quality and readability

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

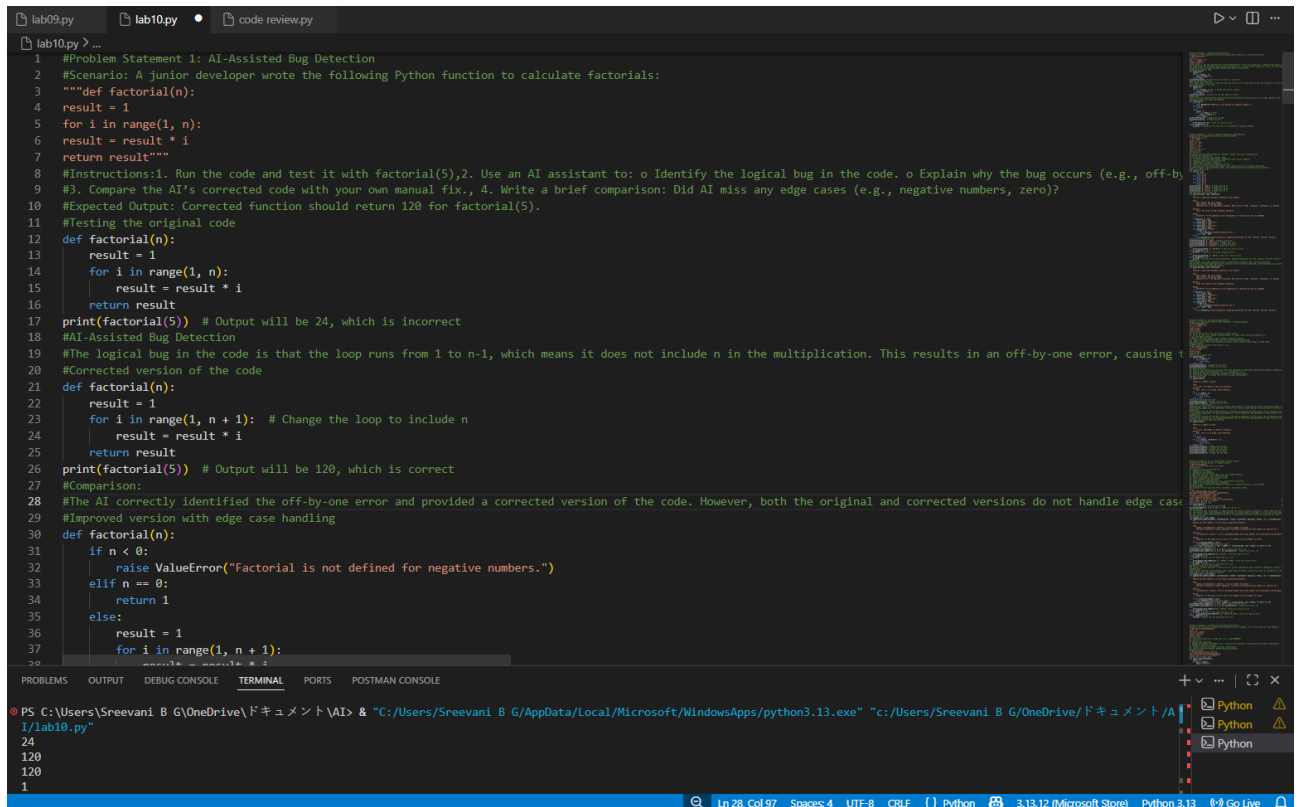
Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

• Screenshot of Generated Code:



```
1 #Problem Statement 1: AI-Assisted Bug Detection
2 #Scenario: A junior developer wrote the following Python function to calculate factorials:
3 """def factorial(n):
4     result = 1
5     for i in range(1, n):
6         result = result * i
7     return result"""
8 #Instructions:1. Run the code and test it with factorial(5).2. Use an AI assistant to: o Identify the logical bug in the code. o Explain why the bug occurs (e.g., off-by-
9 #3. Compare the AI's corrected code with your own manual fix., 4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?
10 #Expected Output: Corrected function should return 120 for factorial(5).
11 #Testing the original code
12 def factorial(n):
13     result = 1
14     for i in range(1, n):
15         result = result * i
16     return result
17 print(factorial(5)) # Output will be 24, which is incorrect
18 #AI-Assisted Bug Detection
19 #The logical bug in the code is that the loop runs from 1 to n-1, which means it does not include n in the multiplication. This results in an off-by-one error, causing t
20 #Corrected version of the code
21 def factorial(n):
22     result = 1
23     for i in range(1, n + 1): # Change the loop to include n
24         result = result * i
25     return result
26 print(factorial(5)) # Output will be 120, which is correct
27 #Comparison:
28 #The AI correctly identified the off-by-one error and provided a corrected version of the code. However, both the original and corrected versions do not handle edge case
29 #Improved version with edge case handling
30 def factorial(n):
31     if n < 0:
32         raise ValueError("Factorial is not defined for negative numbers.")
33     elif n == 0:
34         return 1
35     else:
36         result = 1
37         for i in range(1, n + 1):
38             result = result * i
39     return result
```

Problem Statement 2: Improving Readability & Documentation

Scenario:The following code works but is poorly written:

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
```

Instructions:

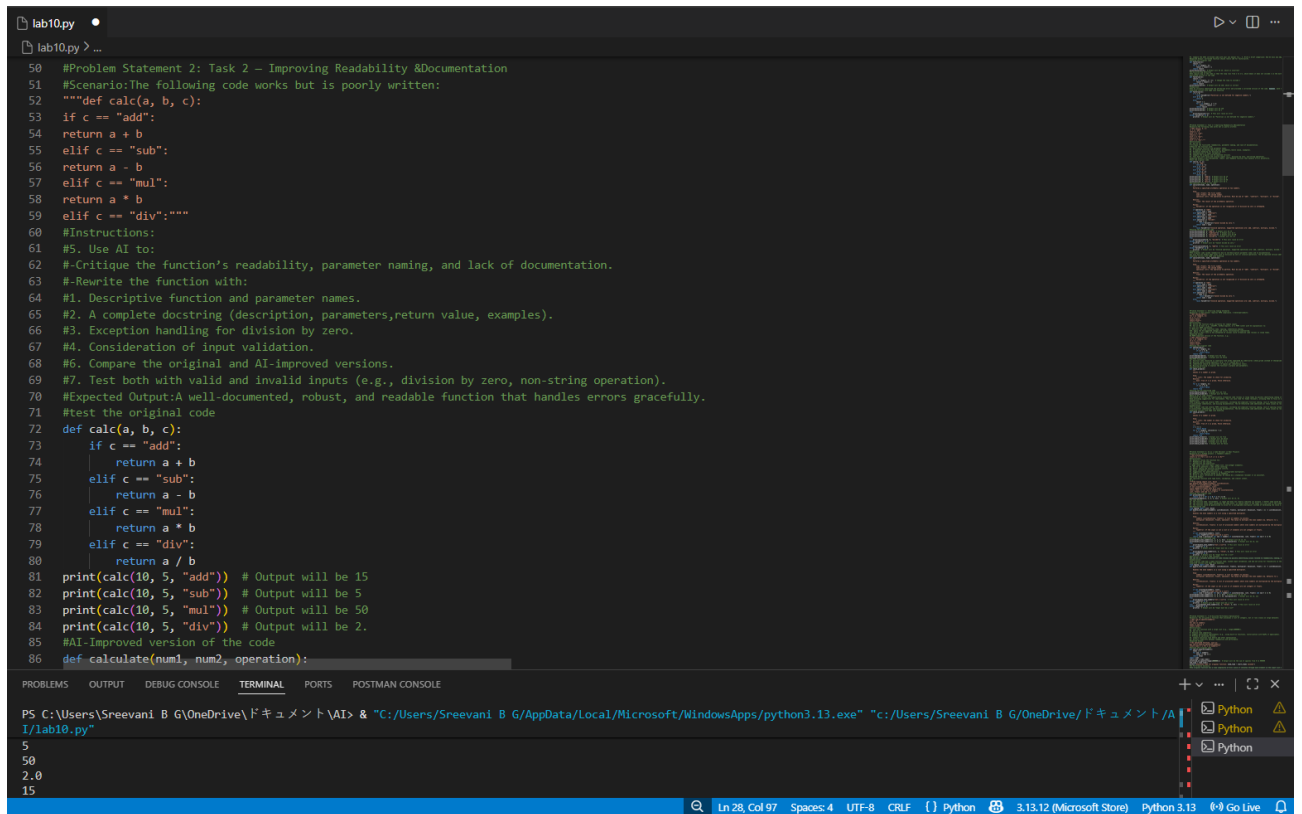
5. Use AI to:

- o Critique the function's readability, parameter naming, and lack of documentation.
- o Rewrite the function with:

1. Descriptive function and parameter names.
2. A complete docstring (description, parameters, return value, examples).
3. Exception handling for division by zero.
4. Consideration of input validation.
6. Compare the original and AI-improved versions.
7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:A well-documented, robust, and readable function that handles errors gracefully.

• Screenshot of Generated Code:



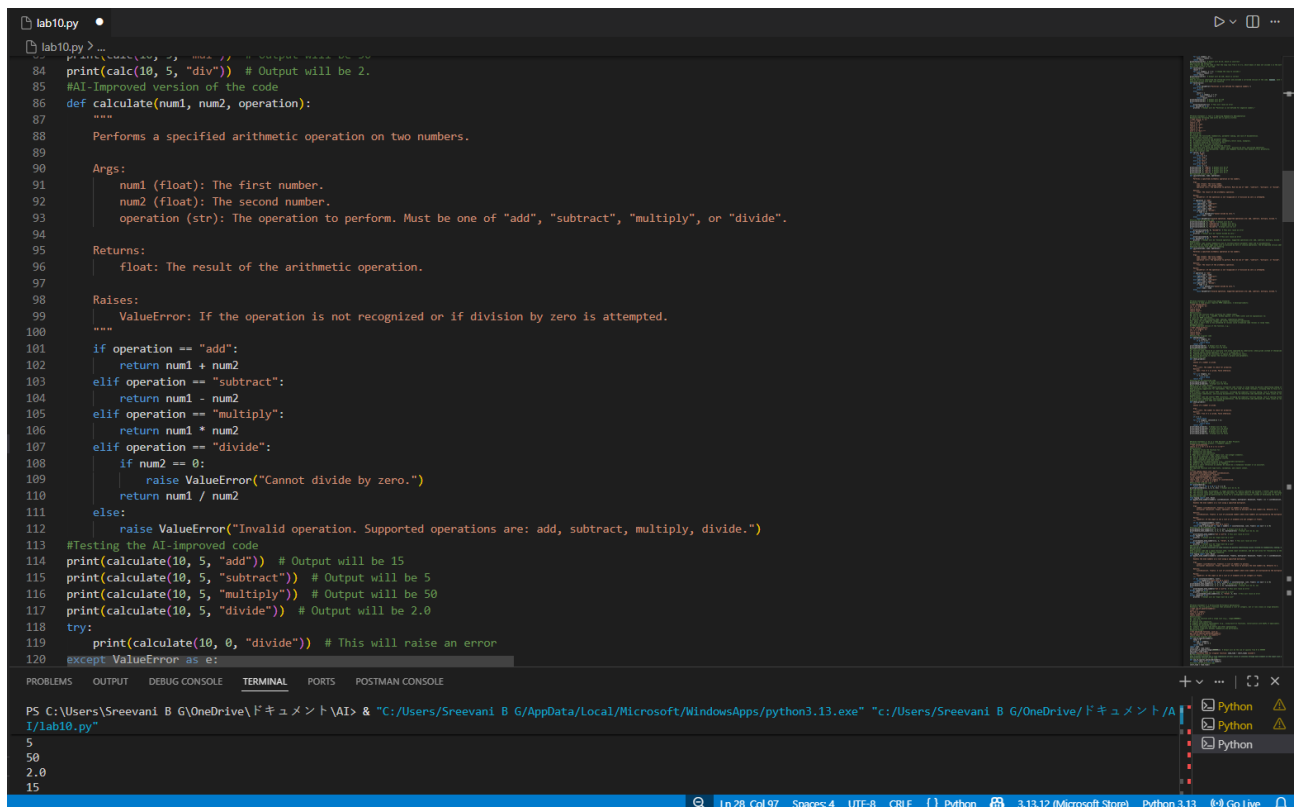
```
lab10.py
lab10.py > ...
50 #Problem Statement 2: Task 2 – Improving Readability & Documentation
51 #Scenario:The following code works but is poorly written:
52 """def calc(a, b, c):
53     if c == "add":
54         return a + b
55     elif c == "sub":
56         return a - b
57     elif c == "mul":
58         return a * b
59     elif c == "div":"""
60 #Instructions:
61 #5. Use AI to:
62 #-Critique the function's readability, parameter naming, and lack of documentation.
63 #-Rewrite the function with:
64 #1. Descriptive function and parameter names.
65 #2. A complete docstring (description, parameters,return value, examples).
66 #3. Exception handling for division by zero.
67 #4. Consideration of input validation.
68 #6. Compare the original and AI-improved versions.
69 #7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).
70 #Expected Output:A well-documented, robust, and readable function that handles errors gracefully.
71 #test the original code
72 def calc(a, b, c):
73     if c == "add":
74         return a + b
75     elif c == "sub":
76         return a - b
77     elif c == "mul":
78         return a * b
79     elif c == "div":
80         return a / b
81 print(calc(10, 5, "add")) # Output will be 15
82 print(calc(10, 5, "sub")) # Output will be 5
83 print(calc(10, 5, "mul")) # Output will be 50
84 print(calc(10, 5, "div")) # Output will be 2.
85 #AI-Improved version of the code
86 def calculate(num1, num2, operation):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"

5
50
2.0
15

Ln 28, Col 97 Spaces: 4 UTF-8 CRLF Python 3.13.12 (Microsoft Store) Python 3.13 Go Live



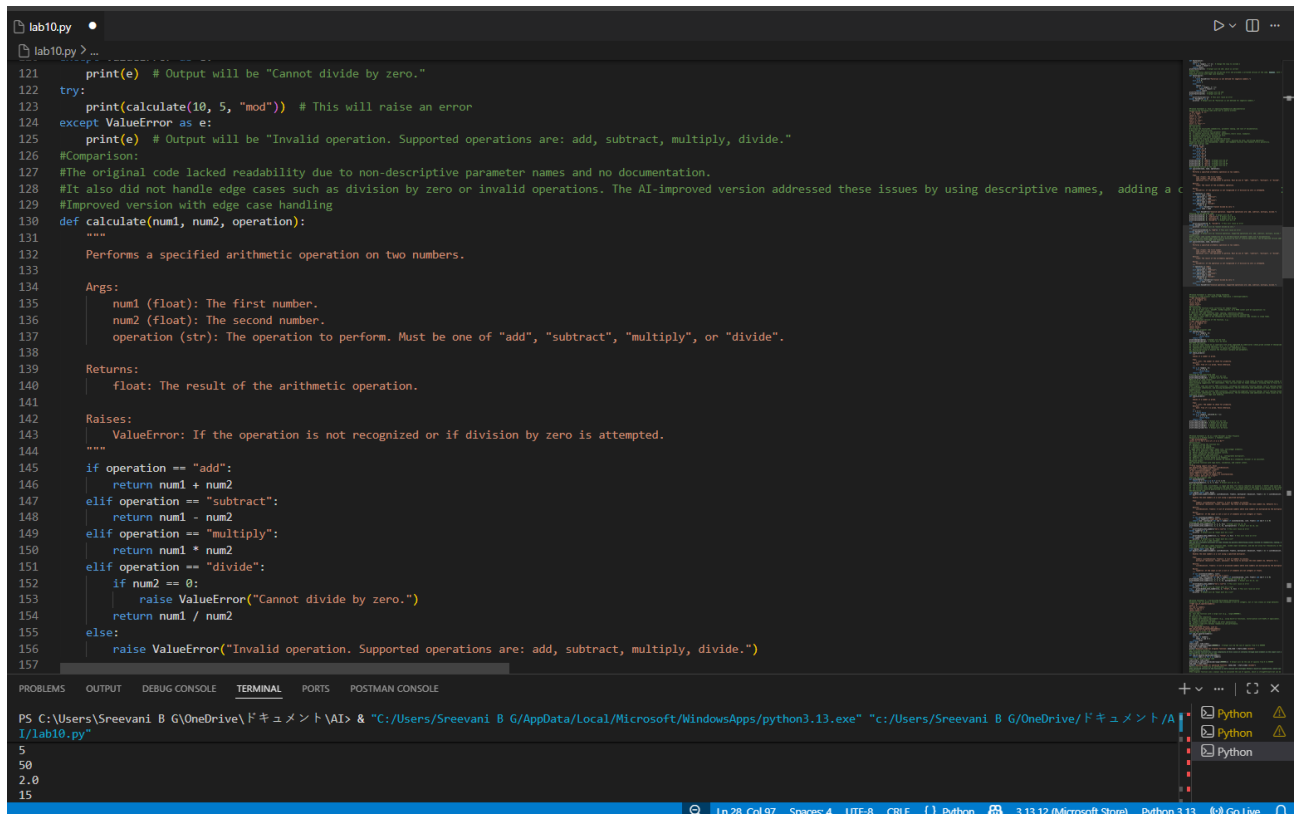
```
lab10.py
lab10.py > ...
84 print(calc(10, 5, "div")) # Output will be 2.
85 #AI-Improved version of the code
86 def calculate(num1, num2, operation):
87     """
88     Performs a specified arithmetic operation on two numbers.
89
90     Args:
91         num1 (float): The first number.
92         num2 (float): The second number.
93         operation (str): The operation to perform. Must be one of "add", "subtract", "multiply", or "divide".
94
95     Returns:
96         float: The result of the arithmetic operation.
97
98     Raises:
99         ValueError: If the operation is not recognized or if division by zero is attempted.
100
101     """
102     if operation == "add":
103         return num1 + num2
104     elif operation == "subtract":
105         return num1 - num2
106     elif operation == "multiply":
107         return num1 * num2
108     elif operation == "divide":
109         if num2 == 0:
110             raise ValueError("Cannot divide by zero.")
111         return num1 / num2
112     else:
113         raise ValueError("Invalid operation. Supported operations are: add, subtract, multiply, divide.")
114 #Testing the AI-improved code
115 print(calculate(10, 5, "add")) # Output will be 15
116 print(calculate(10, 5, "subtract")) # Output will be 5
117 print(calculate(10, 5, "multiply")) # Output will be 50
118 print(calculate(10, 5, "divide")) # Output will be 2.0
119 try:
120     print(calculate(10, 0, "divide")) # This will raise an error
121 except ValueError as e:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"

5
50
2.0
15

Ln 28, Col 97 Spaces: 4 UTF-8 CRLF Python 3.13.12 (Microsoft Store) Python 3.13 Go Live



```
lab10.py
lab10.py
121 print(e) # Output will be "Cannot divide by zero."
122 try:
123     print(calculate(10, 5, "mod")) # This will raise an error
124 except ValueError as e:
125     print(e) # Output will be "Invalid operation. Supported operations are: add, subtract, multiply, divide."
126 #Comparison:
127 #The original code lacked readability due to non-descriptive parameter names and no documentation.
128 #It also did not handle edge cases such as division by zero or invalid operations. The AI-improved version addressed these issues by using descriptive names, adding a docstring, and handling edge cases.
129 #Improved version with edge case handling
130 def calculate(num1, num2, operation):
131     """
132     Performs a specified arithmetic operation on two numbers.
133
134     Args:
135         num1 (float): The first number.
136         num2 (float): The second number.
137         operation (str): The operation to perform. Must be one of "add", "subtract", "multiply", or "divide".
138
139     Returns:
140         float: The result of the arithmetic operation.
141
142     Raises:
143         ValueError: If the operation is not recognized or if division by zero is attempted.
144     """
145     if operation == "add":
146         return num1 + num2
147     elif operation == "subtract":
148         return num1 - num2
149     elif operation == "multiply":
150         return num1 * num2
151     elif operation == "divide":
152         if num2 == 0:
153             raise ValueError("Cannot divide by zero.")
154         return num1 / num2
155     else:
156         raise ValueError("Invalid operation. Supported operations are: add, subtract, multiply, divide.")
157
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:\Users\Sreevani B G\AppData\Local\Microsoft\WindowsApps\python3.13.exe" "C:\Users\Sreevani B G\OneDrive\ドキュメント\AI\lab10.py"

5
50
2.0
15

Ln 28, Col 97 Spaces 4 UTF-8 CRLF Python 3.13.12 (Microsoft Store) Python 3.13 Go Live

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
 - o List all PEP8 violations.
 - o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
for i in range(2, n):
if n % i == 0:
return False
```

return True

• **Screenshot of Generated Code(V1):**

• **Screenshot of Generated Code(V2):**

```
lab10.py
lab10.py > ...

162 #Problem Statement 3: Enforcing Coding Standards
163 #Scenario: A team project requires PEP8 compliance. A developersubmits:
164 """def Checkprime(n):
165     for i in range(2, n):
166         if n % i == 0:
167             return False
168         return True"""
169 #Instructions:
170 #8. Verify the function works correctly for sample inputs.
171 #9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
172 # -List all PEP8 violations.
173 # -Refactor the code (function name, spacing, indentation,naming).
174 #10. Apply the AI-suggested changes and verify functionality ispreserved.
175 #11. Write a short note on how automated AI reviews could streamline code reviews in large teams.
176 #Expected Output:
177 #A PEP8-compliant version of the function, e.g.:
178 """def check_prime(n):
179     for i in range(2, n):
180         if n % i == 0:
181             return False
182         return True"""
183 #Testing the original code
184 def Checkprime(n):
185     for i in range(2, n):
186         if n % i == 0:
187             return False
188     return True
189 print(Checkprime(5)) # Output will be True
190 print(Checkprime(10)) # Output will be False
191 #AI-PEP8 Violations:
192 #1. Function name should be in lowercase with words separated by underscores (check_prime instead of Checkprime).
193 #2. Missing spaces around operators (e.g., n % i should be n % i).
194 #3. Indentation should be consistent (4 spaces per indentation level).
195 #4. Missing docstring to explain the function's purpose and parameters.
196 #AI-Refactored code
197 def check_prime(n):
198     """
199     Checks if a number is prime.
200
201     Args:
202         n (int): The number to check for primality.
203     Returns:
204         bool: True if n is prime, False otherwise.
205     """
206     for i in range(2, n):
207         if n % i == 0:
208             return False
209     return True
210 #Testing the AI-refactored code
211 print(check_prime(5)) # Output will be True
212 print(check_prime(10)) # Output will be False
213 #Note on AI in code reviews:
214 #Automated AI reviews can significantly streamline code reviews in large teams by quickly identifying coding standard violations
215 #and providing suggestions for improvement. This can save time for human reviewers, allowing them to focus on more complex issues such as logic errors and architectural
216 #Comparison:
217 #The original code had several PEP8 violations, including non-compliant function naming, lack of spacing around operators,
218 #inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues by renaming the function to follow PEP8 guidelines, adding space
219 #comparison:
220 #The original code had several PEP8 violations, including non-compliant function naming, lack of spacing around operators,
221 #inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues by renaming the function to follow PEP8 guidelines, adding space
222 #improved version with edge case handling
223 def check_prime(n):
224     """
225     Checks if a number is prime.
226
227     Args:
228         n (int): The number to check for primality.
229     Returns:
230         bool: True if n is prime, False otherwise.
231     """
232     if n <= 1:
233         return False
234     for i in range(2, n):
235         if n % i == 0:
236             return False
237     return True
```

PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"

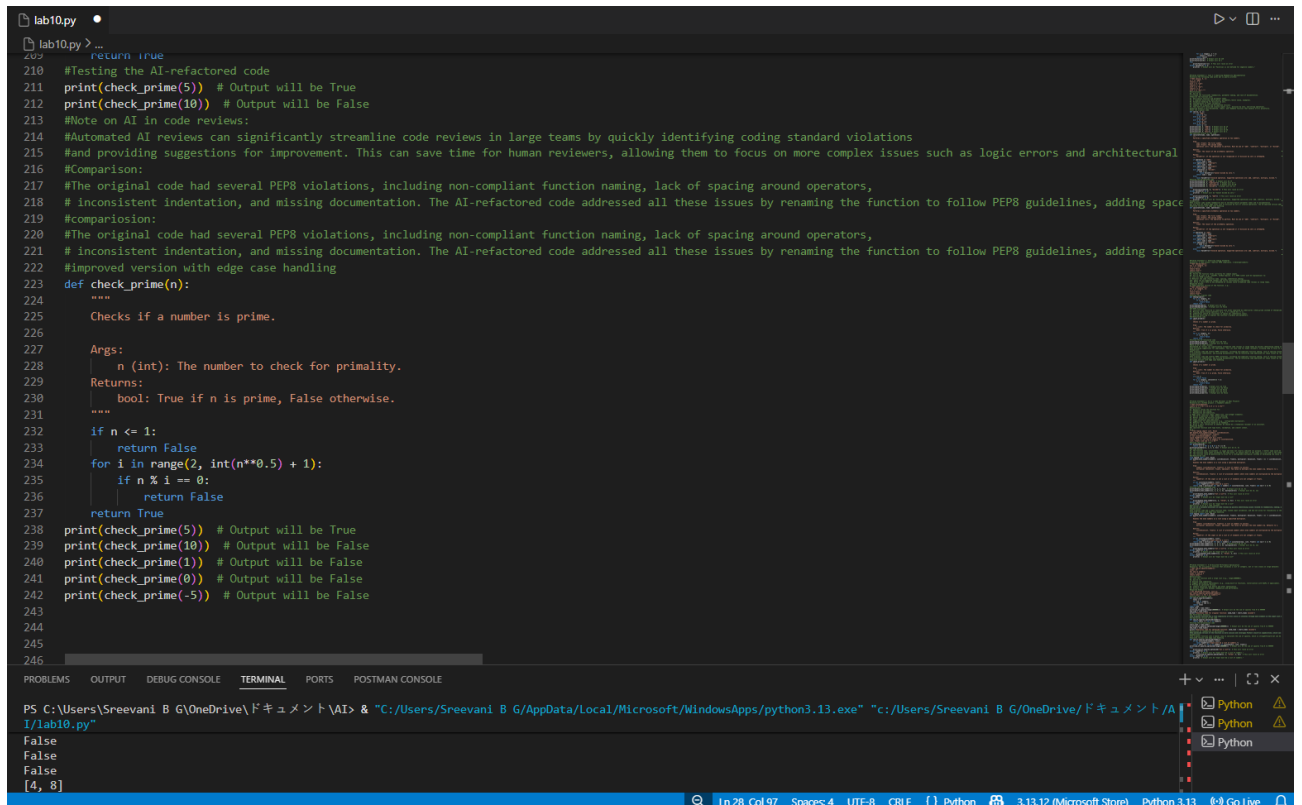
True
False
True
False

```
lab10.py
lab10.py > ...

197 def check_prime(n):
198     """
199     Checks if a number is prime.
200
201     Args:
202         n (int): The number to check for primality.
203     Returns:
204         bool: True if n is prime, False otherwise.
205     """
206     for i in range(2, n):
207         if n % i == 0:
208             return False
209     return True
210 #Testing the AI-refactored code
211 print(check_prime(5)) # Output will be True
212 print(check_prime(10)) # Output will be False
213 #Note on AI in code reviews:
214 #Automated AI reviews can significantly streamline code reviews in large teams by quickly identifying coding standard violations
215 #and providing suggestions for improvement. This can save time for human reviewers, allowing them to focus on more complex issues such as logic errors and architectural
216 #Comparison:
217 #The original code had several PEP8 violations, including non-compliant function naming, lack of spacing around operators,
218 #inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues by renaming the function to follow PEP8 guidelines, adding space
219 #comparison:
220 #The original code had several PEP8 violations, including non-compliant function naming, lack of spacing around operators,
221 #inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues by renaming the function to follow PEP8 guidelines, adding space
222 #improved version with edge case handling
223 def check_prime(n):
224     """
225     Checks if a number is prime.
226
227     Args:
228         n (int): The number to check for primality.
229     Returns:
230         bool: True if n is prime, False otherwise.
231     """
232     if n <= 1:
233         return False
234     for i in range(2, n):
235         if n % i == 0:
236             return False
237     return True
```

PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"

True
False
False
False



```
lab10.py
207 return True
210 #Testing the AI-refactored code
211 print(check_prime(5)) # Output will be True
212 print(check_prime(10)) # Output will be False
213 #Note on AI in code reviews:
214 #Automated AI reviews can significantly streamline code reviews in large teams by quickly identifying coding standard violations
215 #and providing suggestions for improvement. This can save time for human reviewers, allowing them to focus on more complex issues such as logic errors and architectural
216 #Comparison:
217 #The original code had several PEP8 violations, including non-compliant function naming, lack of spacing around operators,
218 #inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues by renaming the function to follow PEP8 guidelines, adding space
219 #comparison:
220 #The original code had several PEP8 violations, including non-compliant function naming, lack of spacing around operators,
221 #inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues by renaming the function to follow PEP8 guidelines, adding space
222 #Improved version with edge case handling
223 def check_prime(n):
224     """
225     Checks if a number is prime.
226
227     Args:
228         n (int): The number to check for primality.
229     Returns:
230         bool: True if n is prime, False otherwise.
231     """
232     if n <= 1:
233         return False
234     for i in range(2, int(n**0.5) + 1):
235         if n % i == 0:
236             return False
237     return True
238 print(check_prime(5)) # Output will be True
239 print(check_prime(10)) # Output will be False
240 print(check_prime(1)) # Output will be False
241 print(check_prime(0)) # Output will be False
242 print(check_prime(-5)) # Output will be False
243
244
245
246
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "C:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"

False
False
False
False
[4, 8]

Ln 28, Col 97 Spaces: 4 UTF-8 CRLF Python 3.13.12 (Microsoft Store) Python 3.13 Go Live

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:In a GitHub project, a teammate submits:

```
def processData(d):  
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

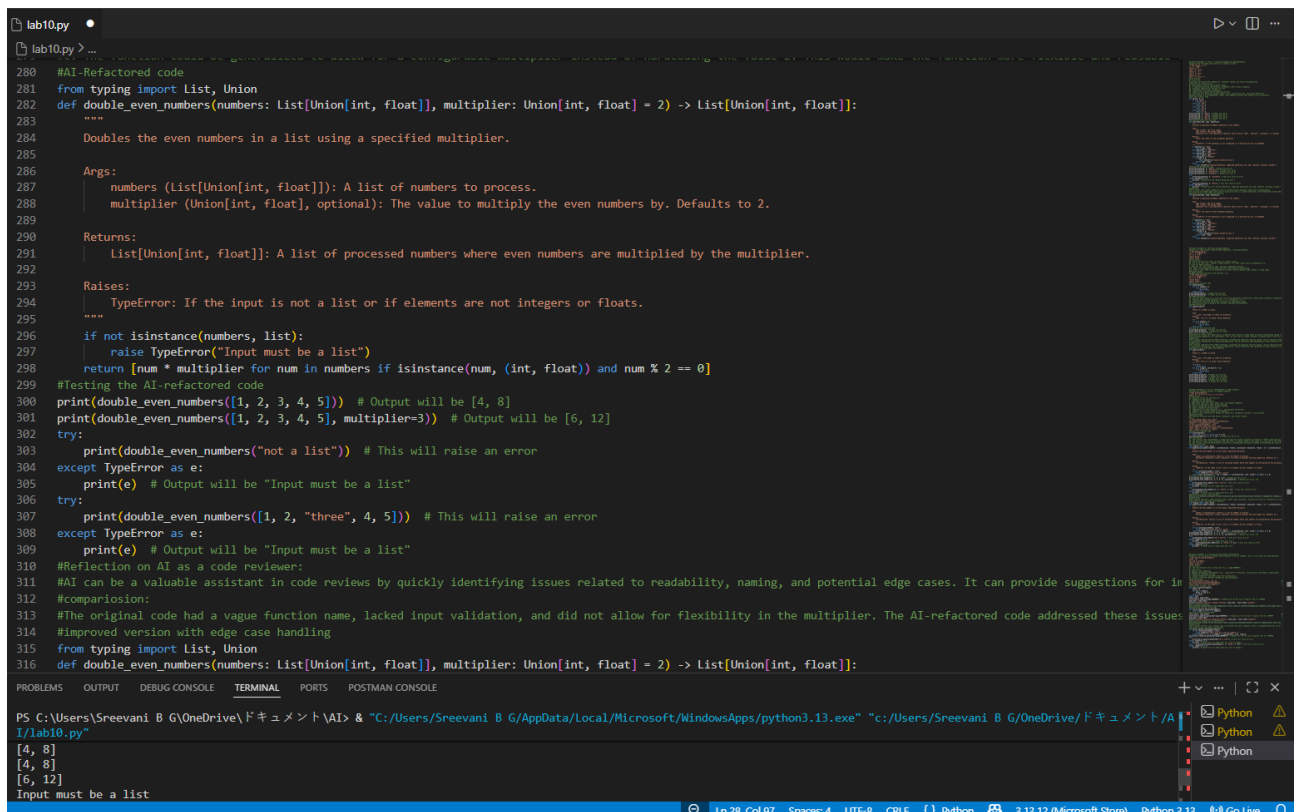
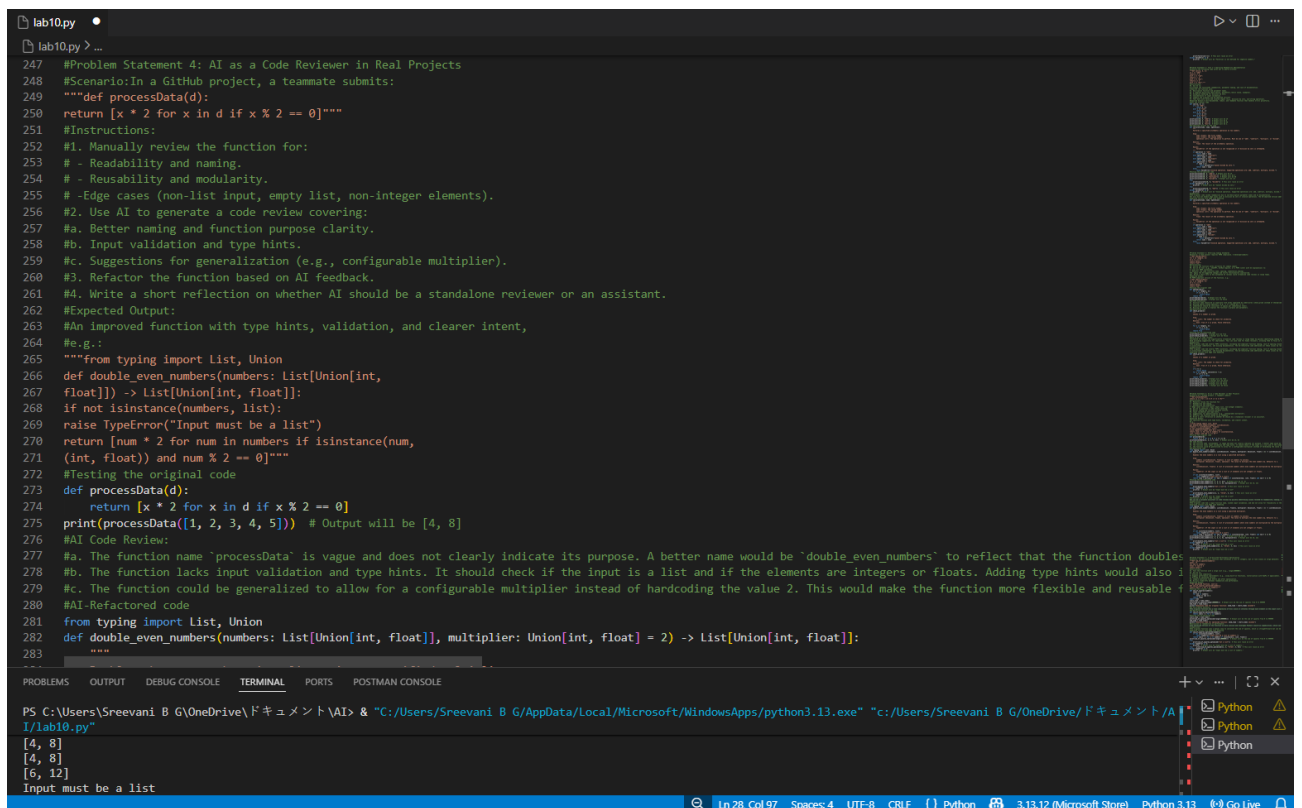
1. Manually review the function for:
 - o Readability and naming.
 - o Reusability and modularity.
 - o Edge cases (non-list input, empty list, non-integers).
2. Use AI to generate a code review covering:
 - a. Better naming and function purpose clarity.
 - b. Input validation and type hints.
 - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

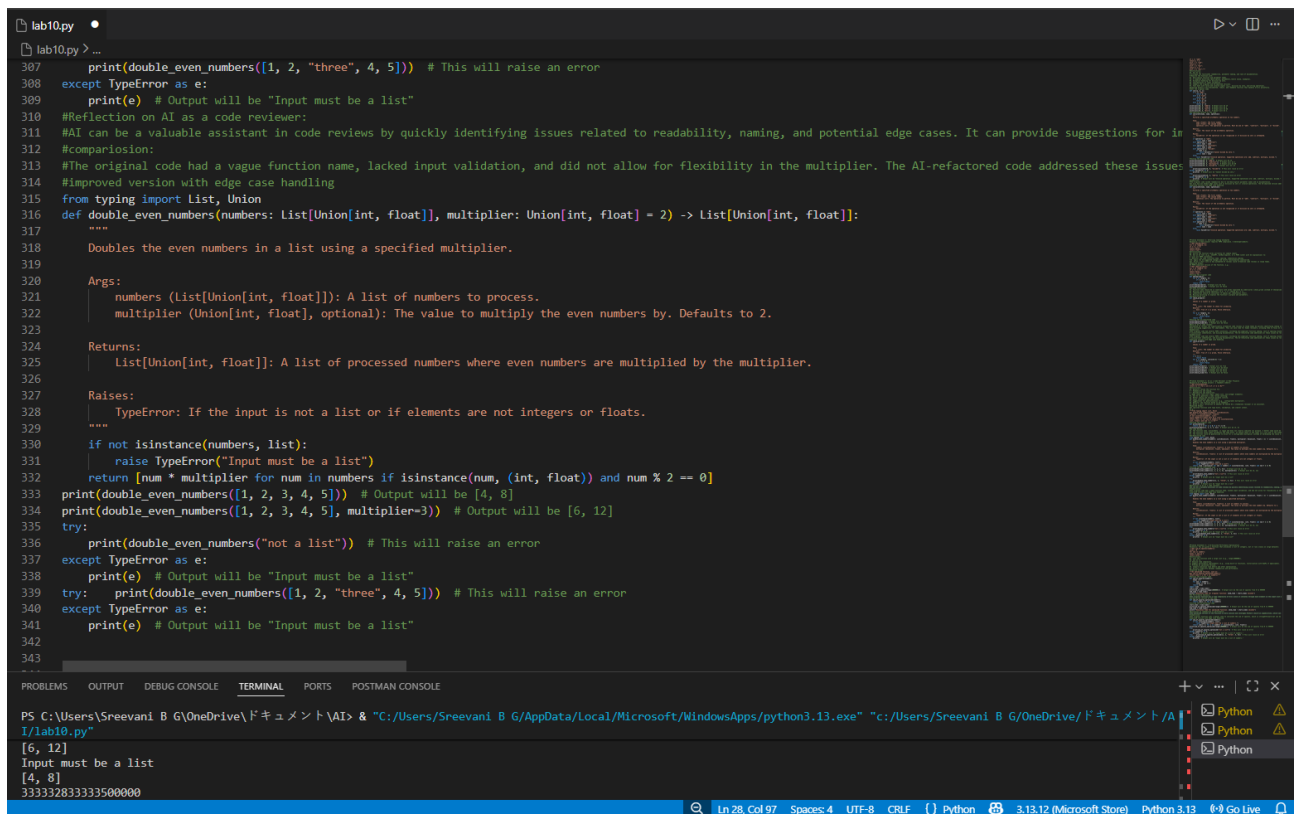
Expected Output:An improved function with type hints, validation, and clearer intent,

e.g.:from typing import List, Union

```
def double_even_numbers(numbers: List[Union[int,  
float]]) -> List[Union[int, float]]:  
    if not isinstance(numbers, list):  
        raise TypeError("Input must be a list")  
    return [num * 2 for num in numbers if isinstance(num,  
(int, float)) and num % 2 == 0]
```

- **Screenshot of Generated Code**





```
lab10.py
lab10.py > ...
307 print(double_even_numbers([1, 2, "three", 4, 5])) # This will raise an error
308 except TypeError as e:
309     print(e) # Output will be "Input must be a list"
310 #Reflection on AI as a code reviewer:
311 #AI can be a valuable assistant in code reviews by quickly identifying issues related to readability, naming, and potential edge cases. It can provide suggestions for in
312 #comparison:
313 #The original code had a vague function name, lacked input validation, and did not allow for flexibility in the multiplier. The AI-refactored code addressed these issues
314 #improved version with edge case handling
315 from typing import List, Union
316 def double_even_numbers(numbers: List[Union[int, float]], multiplier: Union[int, float] = 2) -> List[Union[int, float]]:
317     """
318     Doubles the even numbers in a list using a specified multiplier.
319
320     Args:
321         numbers (List[Union[int, float]]): A list of numbers to process.
322         multiplier (Union[int, float], optional): The value to multiply the even numbers by. Defaults to 2.
323
324     Returns:
325         List[Union[int, float]]: A list of processed numbers where even numbers are multiplied by the multiplier.
326
327     Raises:
328         TypeError: If the input is not a list or if elements are not integers or floats.
329     """
330     if not isinstance(numbers, list):
331         raise TypeError("Input must be a list")
332     return [num * multiplier for num in numbers if isinstance(num, (int, float)) and num % 2 == 0]
333 print(double_even_numbers([1, 2, 3, 4, 5])) # Output will be [4, 8]
334 print(double_even_numbers([1, 2, 3, 4, 5], multiplier=3)) # Output will be [6, 12]
335 try:
336     print(double_even_numbers("not a list")) # This will raise an error
337 except TypeError as e:
338     print(e) # Output will be "Input must be a list"
339 try: print(double_even_numbers([1, 2, "three", 4, 5])) # This will raise an error
340 except TypeError as e:
341     print(e) # Output will be "Input must be a list"
342
343
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
[6, 12]
Input must be a list
[4, 8]
33332833333500000
Ln 28, Col 97 Spaces: 4 UTF-8 CRLF Python 3.13.12 (Microsoft Store) Python 3.13 Go Live
```

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

Instructions:

1. Test the function with a large list (e.g., `range(1000000)`).
2. Use AI to:
 - o Analyze time complexity.
 - o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
 - o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)
```

• Screenshot of Generated Code


```
lab10.py •
lab10.py > ...
346 #Problem Statement 5: – AI-Assisted Performance Optimization
347 #Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:
348 """def sum_of_squares(numbers):
349     total = 0
350     for num in numbers:
351         total += num ** 2
352     return total"""
353 #Instructions:
354 #1. Test the function with a large list (e.g., range(1000000)).
355 #2. Use AI to:
356 #    - Analyze time complexity.
357 #    - Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
358 #    - Provide an optimized version.
359 #3. Compare execution time before and after optimization.
360 #4. Discuss trade-offs between readability and performance.
361 #Expected Output:
362 """An optimized function, such as:
363 def sum_of_squares_optimized(numbers):
364     return sum(x * x for x in numbers)"""
365 #Testing the original code
366 def sum_of_squares(numbers):
367     total = 0
368     for num in numbers:
369         total += num ** 2
370     return total
371 import time
372 start_time = time.time()
373 print(sum_of_squares(range(1000000))) # Output will be the sum of squares from 0 to 999999
374 end_time = time.time()
375 print(f"Execution time for original function: {end_time - start_time} seconds")
376 #AI Performance Analysis:
377 #The original function has a time complexity of O(n) since it iterates through each element in the input list once. However, the use of a loop and manual accumulation can be inefficient for large datasets.
378 #AI-Optimized version of the code
379 def sum_of_squares_optimized(numbers):
380     return sum(x * x for x in numbers)
381 #Testing the AI-optimized code
382 start_time = time.time()
383 print(sum_of_squares_optimized(range(1000000))) # Output will be the sum of squares from 0 to 999999
384 end_time = time.time()
385 print(f"Execution time for optimized function: {end_time - start_time} seconds")
386 #Discussion on readability vs performance:
387 #The optimized version of the function is more concise and leverages Python's built-in capabilities, which can lead to improved performance, especially on large datasets.
388 #Comparison:
389 #The original function uses a manual loop to calculate the sum of squares, which is straightforward but can be inefficient for large datasets. The optimized version uses a generator expression, which is more concise and efficient.
390 #Improved version with edge case handling
391 def sum_of_squares_optimized(numbers):
392     if not isinstance(numbers, list):
393         raise TypeError("Input must be a list of numbers.")
394     return sum(x * x for x in numbers if isinstance(x, (int, float)))
395 print(sum_of_squares_optimized(range(1000000))) # Output will be the sum of squares from 0 to 999999
396 try:
397     print(sum_of_squares_optimized("not a list")) # This will raise an error
398 except TypeError as e:
399     print(e) # Output will be "Input must be a list of numbers."
400 try:
401     print(sum_of_squares_optimized([1, 2, "three", 4, 5])) # This will raise an error
402 except TypeError as e:
403     print(e) # Output will be "Input must be a list of numbers."
404
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
File "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py", line 387, in sum_of_squares_optimized
    raise TypeError("Input must be a list of numbers.")
TypeError: Input must be a list of numbers.
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI>
```

```
lab10.py •
lab10.py > ...
378 #AI-Optimized version of the code
379 def sum_of_squares_optimized(numbers):
380     return sum(x * x for x in numbers)
381 #Testing the AI-optimized code
382 start_time = time.time()
383 print(sum_of_squares_optimized(range(1000000))) # Output will be the sum of squares from 0 to 999999
384 end_time = time.time()
385 print(f"Execution time for optimized function: {end_time - start_time} seconds")
386 #Discussion on readability vs performance:
387 #The optimized version of the function is more concise and leverages Python's built-in capabilities, which can lead to improved performance, especially on large datasets.
388 #Comparison:
389 #The original function uses a manual loop to calculate the sum of squares, which is straightforward but can be inefficient for large datasets. The optimized version uses a generator expression, which is more concise and efficient.
390 #Improved version with edge case handling
391 def sum_of_squares_optimized(numbers):
392     if not isinstance(numbers, list):
393         raise TypeError("Input must be a list of numbers.")
394     return sum(x * x for x in numbers if isinstance(x, (int, float)))
395 print(sum_of_squares_optimized(range(1000000))) # Output will be the sum of squares from 0 to 999999
396 try:
397     print(sum_of_squares_optimized("not a list")) # This will raise an error
398 except TypeError as e:
399     print(e) # Output will be "Input must be a list of numbers."
400 try:
401     print(sum_of_squares_optimized([1, 2, "three", 4, 5])) # This will raise an error
402 except TypeError as e:
403     print(e) # Output will be "Input must be a list of numbers."
404
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
File "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py", line 387, in sum_of_squares_optimized
    raise TypeError("Input must be a list of numbers.")
TypeError: Input must be a list of numbers.
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI>
```